

Open in app ↗



Medium



Search



Write



Web3 Foundation

Polkadot's Messaging Scheme



Web3 Foundation Team

Follow

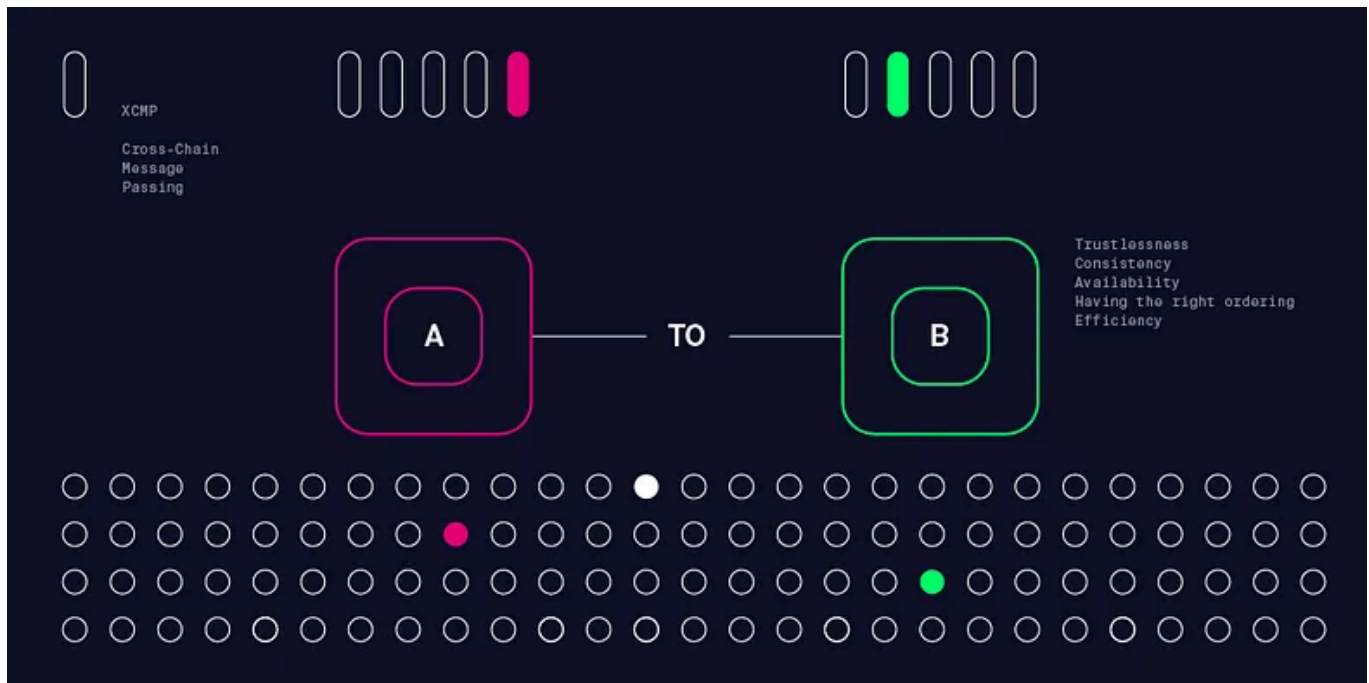
11 min read · Jan 13, 2020



776



2



Authors: Fatemeh Shirazi, Logan Saether, Alistair Stewart, Rob Habermeier, Gavin Wood

The Web3 Foundation Research team has been working the last months on a text outlining the functionality of Cross-Chain Message Passing (XCMP). It's a key component in Polkadot, the Web3 Foundation's flagship protocol. We're excited to share our work with you!

Cross-Chain Message Passing (XCMP) scheme is a subset of the Polkadot protocol. It defines how messages can be passed among parachains with no additional trust assumptions beyond the economic security of the relay chain. This write-up addresses the messaging protocol of parachains and relies heavily on Polkadot's unique relay chain architecture and design.

The protocol covers:

- In terms of consensus: mechanisms for **queuing** and **ordering** of messages.
- In conjunction with the rest of the relay chain and in particular GRANDPA finalization: **data availability**.
- In conjunction with the parachain validation function: **message input and output**.
- In addition, this write-up also reviews **delivery**, how **consistent history** is achieved and ideas for **preventing DoS attacks**. Finally, we review XCMP in conjunction with **SPREE** and conclude by summarizing the properties that are achieved by XCMP.

Message semantics and networking details such as peer discovery are not addressed in this write-up.

Introduction

One of the key features of Polkadot “version 1.0” is to let its otherwise isolated parachains send messages between each other with guarantees and in a secure and trust-free manner.

For the present purposes, we define the term *message* in much the same way as *transaction*. Both refer to data coming from outside of the receiving chain, and both imply and require that the chain act on the data following the chain’s internal logic. Allowing for some level of delay typical for real-world systems, the chain cannot reject or confound the implications of the data. For example, in the Bitcoin context, this property means that a faulty or malicious miner in Bitcoin cannot redirect funds and is thus the foundation of good cryptoeconomic consensus systems.

The key difference between a transaction and a message is that a transaction contains a signature to prove the provenance of the data (and thus the authority of the instructions), whereas with a message the provenance is proven merely by virtue of Polkadot’s internal Byzantine-resistant cryptoeconomic validation infrastructure, in much the same way as Ethereum’s inter-contract message passing.

Example

Before we dive into detail on each component of XCMP, let’s first take an example of how an outbound message on a smart contract parachain (referred to as A in Figure 1) will be networked to the inbound queue of a Decentralized Finance (DeFi) parachain (referred to as B in Figure 1) for inclusion in the next block candidate from the collator of the DeFi parachain.

At the relay chain block 300, the smart contract parachain initiates a message that is targeted toward the endpoint of `32`, which is the Parachain

ID of the DeFi parachain. The message will first be included into the outbound, or egress queue, of the smart contract parachain.

Full nodes of the smart contract parachain will begin gossiping (see section Delivery below) the message in the network. If some nodes of the smart contract chain are also full nodes of the DeFi chain, and these nodes act as the glue between the two gossip networks by relaying the message. If there are no shared nodes of the networks that need to be traversed, then a fallback (see section Fallback below) mechanism is invoked.

Once the message has reached the collator for the DeFi parachain, they take this message (and any other messages it has received) and enter it onto the inbound, or ingress, queue for processing in its next block candidate.

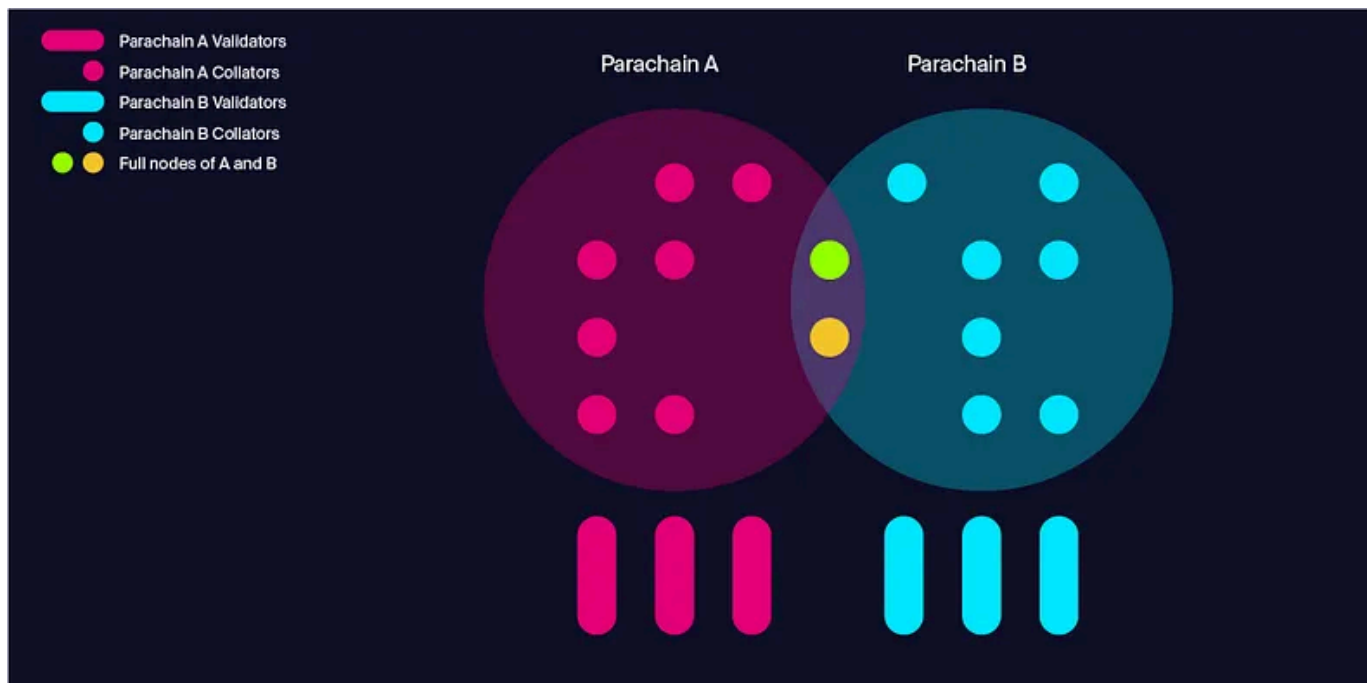


Figure 1: shows two parachains A and B, their corresponding collators and full nodes. There are two nodes that are both full nodes of parachain A network and parachain B network.

The collator on the DeFi parachain will produce a block candidate for relay chain block 301. This block candidate will require proof that the messages it

acted on from A's block were the correct messages. The relay chain block 300 contains a parachain header for A's block, a small amount of data that includes a message root hash that can be used to authenticate messages.

This block candidate will include a relay chain light-client proof that this message root was in the relay chain and combine this with a proof sent with the message by the sending chain.

The parachain validator for the DeFi parachain will be able to use these proofs to validate the integrity of the proposed block candidate from the DeFi parachain. The original message of the smart contract chain is then included in the DeFi parachain *without additional trust assumptions and by relying on the full security of Polkadot*.

Queuing and Ordering Messages

Every parachain block in Polkadot produces a possibly empty list of messages to route to every other block. These are known as *egress queues*. Once a message is routed, it enters a parachain's *ingress queue*. Parachains must process ingress lists *in order*.

A collator or validator seeking to collect messages for the egress queues of a certain parachain invokes ingress for that parachain and searches the propagation pool for the relevant messages, waiting for any that have not been gossiped yet.

Delivering Messages

Let us assume that we have a connected network of full nodes for each parachain. We assume each full node is aware of a subset of other full nodes in the system, which we refer to as neighboring nodes. Note that we do not

make any assumptions about the topology and the diameter of these networks.

The simplest way to send messages is with a gossip protocol. Recall that peers communicate with each other about their view on current leaves constantly. To achieve a more efficient delivery, unrouted messages are only gossiped to neighboring nodes that have the same view.

If there are nodes in common between these two networks, messages will be gossiped from one parachain network to another parachain network.

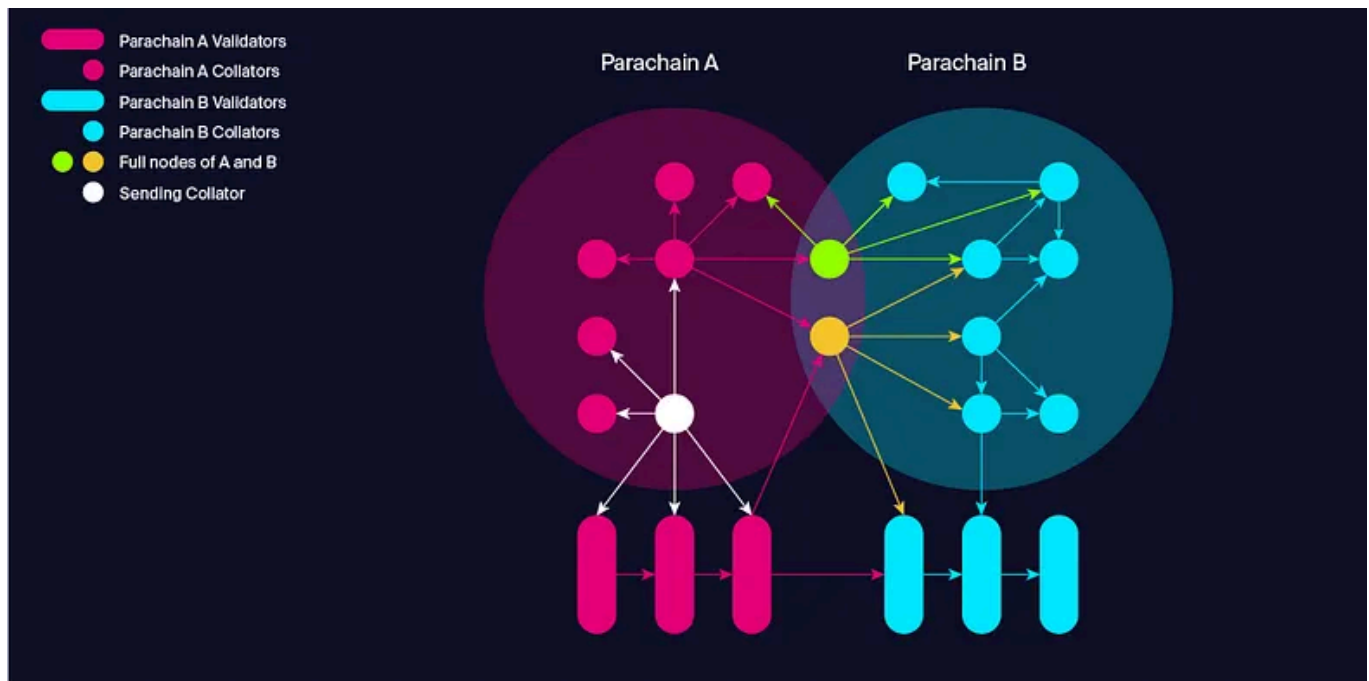


Figure 2: shows the message delivery using gossiping. We assume the message is sent out by the pink sending collator, who produced the latest parachain block.

Fallback Delivery:

However, if the receiving parachain validators realize that the message has not been gossiped in the recipient parachain, they request the message from parachain validators of the sending parachain. Once they have received them, they gossip those messages in the receiving parachain network.

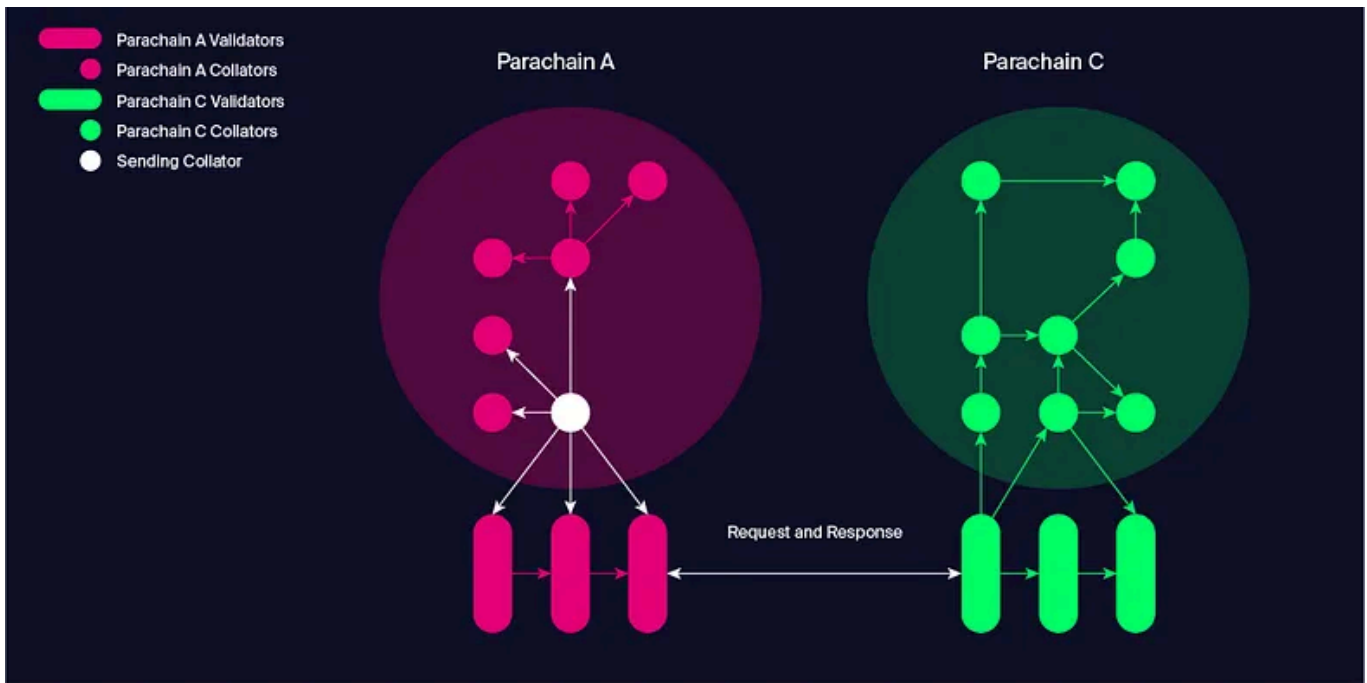


Figure 3: shows the fallback delivery when the sending and receiving parachains do not share any full nodes.

The fallback delivery mechanism is shown in Figure 3, where we assume parachain A wants to send a message to parachain C, with whom it shares no common full node. Once the parachain validators of parachain C notice the message has not arrived, they send a request to the sending parachain validators, who are responsible for holding the egress messages from their parachain. Once the response to their request arrives, the parachain validators of parachain C gossip the message within parachain C.

Getting Consistent History

A key property we want from XCMP is for canonical parachain blocks, i.e. those that we eventually agree have happened. This means, in the current parachain block only to act on those messages that were sent from parachain blocks that are themselves both canonical and earlier than the current parachain block.

The relay chain defines a history for all parachains. For example, a block from parachain B whose header was in relay chain block 301, can say that it

has acted on all messages up to block 300, and if so, it should be acting on messages sent from a parachain block of parachain A if and only if A's parachain block header appeared in relay chain block 300 or earlier.

This means that the relay chain needs to play a role in authenticating messages. However, as we cannot put much data in these parachain headers, the relay chain should not have the message payload itself. Instead, we achieve keeping consistent history efficiently by using nested Merkle trees. The header of a parachain block that corresponds to sent messages will contain a single message root hash, the root of a Merkle tree. In turn, the leaves of this Merkle tree are the head of hash chain of messages from this parachain to another.

This means that there is a sequence of hashes that contains each message hash, allowing the verification of all sent messages from one parachain to another from this one hash. This allows a collator to construct a proof, consisting of many hashes, that they acted on the messages and only those messages that they should be acting on, by first showing that the message root was in the relay chain and then giving a proof that these were the messages from the message root hash.

For more information about this topic please see [here](#).

Input and Output Validation

Recall that Polkadot is comprised of one single relay chain and a number (tentatively up to 100) parachains.

Parachain headers contain a message root of outgoing messages. To produce a parachain block on a parachain that builds on a particular relay chain block, a collator would need to look at which parachain headers were built

between that relay chain block and the relay chain that included the header of the last parachain block of this parachain. For those messages, the parachain needs to act on the corresponding message data.

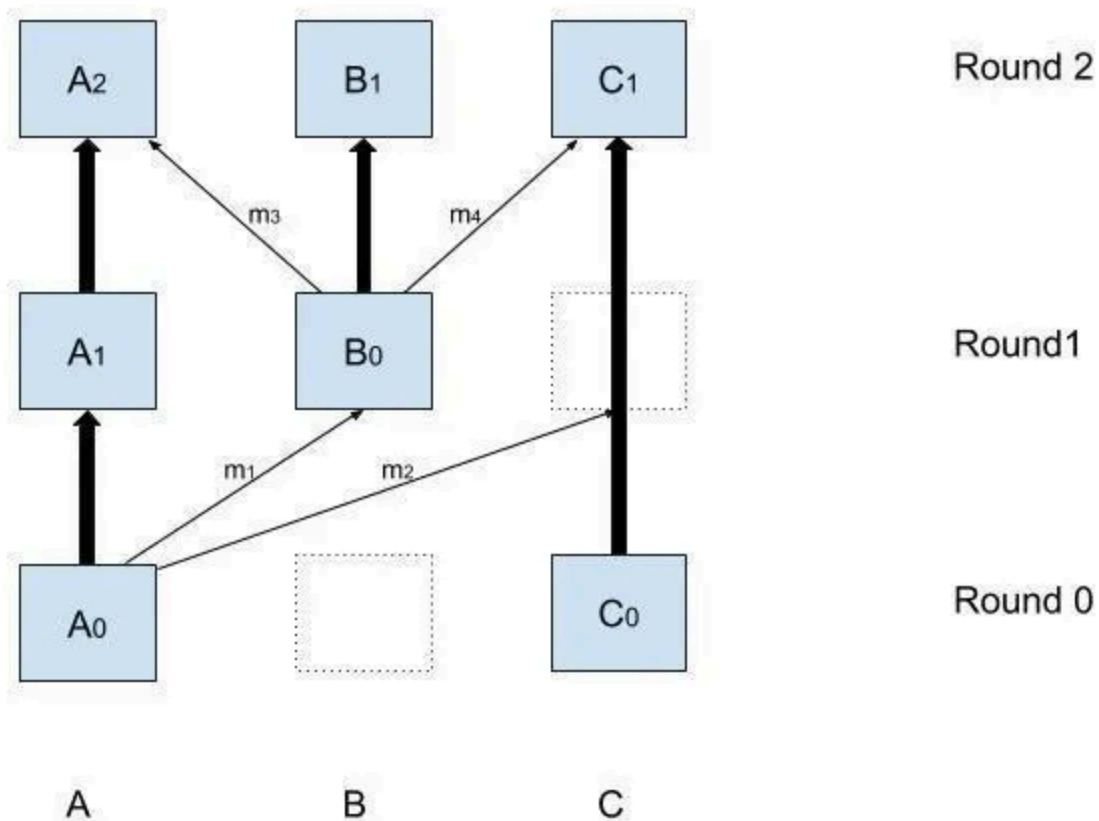


Figure 4: shows the parachain blocks that have been built for three parachains A,B,C in the three rounds 0, 1, 2 and the messages that have been sent in each round among these parachains.

The parachain state-transition verification function (STVF) uses a validation function to verify that input messages are acted on. The validation function is a piece of WebAssembly that checks that the parachain's state-transition is in fact valid. It relates a new state of the parachain and a set of output messages to a digest of the previous state of the parachain, the parachain block data, and a set of input messages that have been faithfully routed from other parachains or the relay chain.

Figure 4 shows an example where the produced parachain blocks and messages among three parachains A,B, and C are shown for rounds 0, 1, 2. Let us assume parachain B does not produce any parachain blocks in round 0, and parachain C does not produce a parachain block in round 1. Parachain block B1 produced in round 1 needs to have taken message m1 as input message and replies to parachain A by sending message m3 at round 1. Parachain block C1 produced in round 2 needs to take messages m2 and m4 in its unprocessed ingress queue.

Availability for Messages

Once the messages have been included in the egress queues, they are kept by collators and full nodes of the sending parachain. When the sending parachain blocks' headers have been included in the relay chain, the parachain validators will also keep the messages. Collators and full nodes of the receiving parachain will also need to be aware of the payload of messages sent among parachains. All other entities who need to know about the existence of the messages may only store hashes, which can be used to authenticate messages.

To guarantee availability, we require that all validators hold erasure-coded pieces that can recover any of the parachain messages. These erasure-coded pieces are produced and distributed by the parachain validators of the sending parachain. 1/3 of these erasure-coded pieces suffice to recover all messages. Finality requires these erasure-coded pieces to have been received by voters (validators), otherwise they will be punished for voting. Thus, 2/3 of erasure-coded pieces must be available once finality is reached; hence, we can guarantee that a finalized message is also available.

Preventing DoS Attacks

Note that the aim of XCMP is not to determine any standard format for messages. However, each parachain has a limit on the total size of messages that it can send to another parachain. Moreover, the gossiping protocol uses a bounding delivery to avoid large overhead.

For parathreads not getting blocks into the relay chain frequently, the queue of unprocessed messages might grow quite substantially. To cap this, the sending parachain will maintain an egress queue for this chain which has a size limit. It can remove old messages only when it knows that they have been received. The receiving chain publishes a watermark stating which block number — and within that, which parachain — it has processed messages up to. The sending chain can use this watermark to prune its egress queue.

Furthermore, we plan to give the receiving parachain the ability to block another parachain from sending messages (*this feature is not implemented yet*). Parathreads may also disable the XCMP function to avoid having to process large amounts of messages.

XCMP and SPREE

Shared Protected Runtime Execution Enclaves (SPREE) are fragments of logic similar to runtime modules but that live on the relay chain and can have their functionality opted into by parachains.

These fragments of logic are blobs of WebAssembly code uploaded onto Polkadot either through a governance mechanism or by parachains. Once the blob is uploaded to Polkadot, all other parachains can decide to opt in to the logic. The SPREE module would retain its own storage independent of the parachain but would be able to be called through an interface with the

parachain. Parachains will send messages to the SPREE module synchronously. For more information about SPREE see its wiki [article](#).

It will be possible to address an XCMP message to a SPREE module and guarantee that when that message is acted on, it will use the same code from that SPREE module as any other parachain. SPREE modules are important to the overall XCMP architecture because they provide a guarantee that a certain interpretation of the code will be executed on the destination parachains. While XCMP guarantees the delivery of a message, it does not guarantee what code will be executed, i.e. how the receiving parachain will interpret the message. Updates to the code of a SPREE module will be simultaneous across parachains. In addition to the security benefits, this means that changing message formats is possible without coordinating updates across many parachains.

In summary, while XCMP accomplishes trustless message passing, SPREE is the trustless interpretation of the message and a key part to the usefulness of XCMP. XCMP messages that are addressed to SPREE modules give the developers and users of the dispatch messages clarity to how the message will be processed.

Summarizing XCMP's Properties

The XCMP scheme achieves the following properties:

Trustlessness Since the same set of validators secure one parachain as another while they also guarantee correct message passing, XCMP requires no more trust than a single blockchain would.

Consistency We provide absolute guarantee that the messages received were exactly those sent, even despite any chain reorgs.

Availability Polkadot guarantees that the messages will not be lost and are kept available. This is achieved by distributing erasure-coded pieces that can be used to reconstruct messages.

Maintaining the right ordering for messages output by parachain blocks is guaranteed by Input/Output validation.

Efficiency The protocol avoids too much bandwidth overhead and allows messages to arrive as quickly as possible.

For more information about Web3 Foundation, please visit web3.foundation. For a deeper dive into Polkadot's functionality and features, check our [Wiki](#), which we are constantly updating and elaborating.

Blockchain

Messaging

Cross Chain Messaging

Blockchain Messaging

Xcmp



Published in Web3 Foundation

3K followers · Last published Oct 13, 2025

Follow

Web 3.0 Technologies Foundation nurtures and stewards technologies and applications in the fields of decentralised web software protocols.



Written by Web3 Foundation Team

5.8K followers · 13 following

Follow

Web3 Foundation is building an internet where users are in control of their own data, identity and destiny. Our primary projects are Polkadot and Kusama.

Responses (2)



Ryc Brownrigg

What are your thoughts?



Tom Nash

Feb 4, 2020



If some nodes of the smart contract chain are also full nodes of the DeFi chain, and these nodes act as the glue between the two gossip networks by relaying the message.

What's the format that this node crossover would take? I'm assuming that you can't just run a couple of separate binaries on the same machine and be a participant in this gossiping, though that may be an incorrect assumption. Is there some... [more](#)



2

[Reply](#)



paul arssov

Aug 20, 2022



The article does not make sense for me

Cross-chain is achieved by 1 program (Dapp) calling the rest api for each of the chains involved and then processing the returns.

In my opinion there is no need of the elaborate schemes described...



[Reply](#)

More from Web3 Foundation Team and Web3 Foundation



 In Web3 Foundation by Web3 Foundation Team

New Grant Guidelines

At the Web3 Foundation, we’re constantly evolving our grant programs to better...

Oct 7  13



 In Web3 Foundation by Web3 Foundation Team

Web3 Foundation to Participate in Polkadot OpenGov Treasury Voting

Effective immediately, the Web3 Foundation will selectively vote on Treasury proposals.

Oct 1  70  3





 In Web3 Foundation by Web3 Foundation Team

Decentralized Voices Program: DV DAO Voting Power to 3M

Web3 Foundation is making a measured update to our Decentralized Voices program...

Oct 3  12



 In Web3 Foundation by Web3 Foundation Team

The Third Cohort of the Decentralized Nodes is a fact!

On September 10, 2025 we launched the Third Cohort of the Decentralized Nodes...

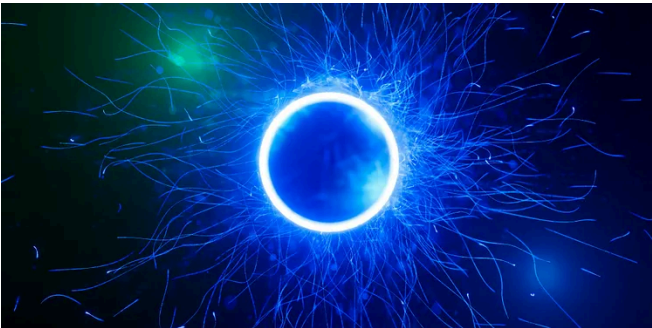
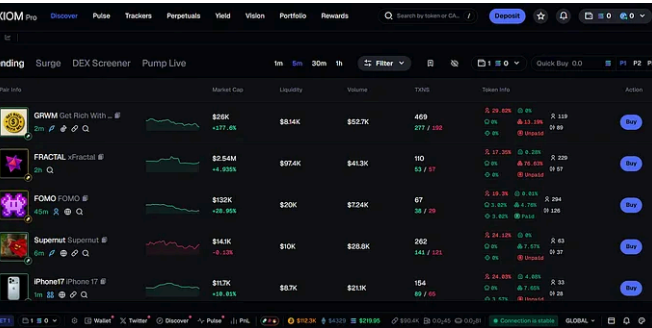
Oct 13  63



See all from Web3 Foundation Team

See all from Web3 Foundation

Recommended from Medium



Dinakar

This company reached \$100M in revenue in just under 4 months

It's not Lovable, Cluely, or Cursor we're talking about

Sep 10



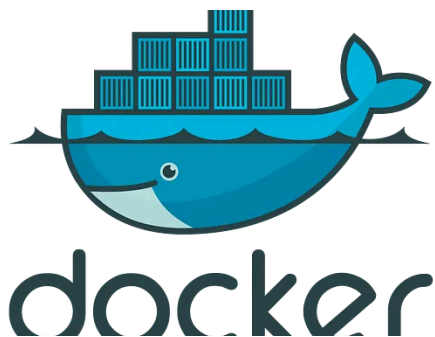
TRON DAO

Justin Sun's Representatives Issue Statement Asserting Bloomberg...

Representatives of Justin Sun issued the following statement on his behalf.

Aug 12

10 1



Abhinav

Docker Is Dead — And It's About Time

Docker changed the game when it launched in 2013, making containers accessible and...

Will Lockett

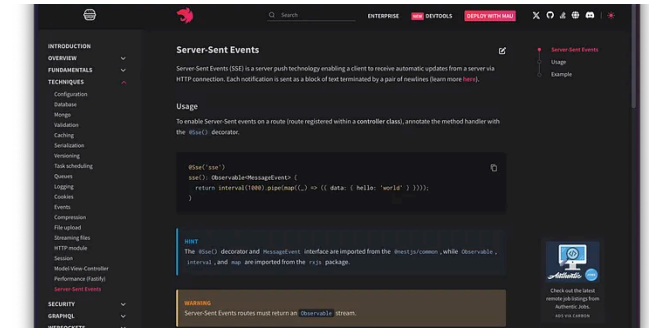
The AI Bubble Is About To Burst, But The Next Bubble Is Already...

Techbros are preparing their latest bandwagon.

Sep 14

14.2K

486



Imtiaz Nayeem Shawon

Real-Time Updates in NestJS with Server-Sent Events (SSE)

In today's world, real-time updates are everywhere — chat apps, stock tickers,...

Sep 17

1



In Angular Blog by Minko Gechev

Announcing Angular v20

The past couple of years have been transformative for Angular, as we've...



Jun 8



7K



199



May 28



2.9K



39



See more recommendations